

NeoBank360 Sprint 5 Work Documentation

Sprint 5 Theme

Sprint 5 focused on making NeoBank360 more enterprise-ready by adding advanced analytics, audit visibility, and deeper user financial intelligence.

The main idea of Sprint 5 was:

```
Previous sprints created banking data.  
Sprint 5 converts that data into business intelligence.
```

Sprint 5 added:

- Advanced Admin Transaction Analytics
- Advanced Admin Loan Analytics
- System Audit Log Dashboard
- Advanced User Spending Analytics
- Budget vs Actual Analytics
- Wealth and Liability Analytics
- Net Worth Timeline
- Reward Growth Tracking
- Loan Payoff Forecast

Day 42: Advanced Admin Analytics - Transactions and Loans

Session 1: Backend Admin Transaction Analytics

Endpoint

```
GET /api/admin/analytics/transactions?timeframe={7d|30d|YTD}
```

This endpoint gives the admin a platform-wide transaction analytics summary.

It is protected with:

```
@PreAuthorize("hasRole('ADMIN')")
```

Only ADMIN users can access this API.

Why This Endpoint Was Needed

The admin dashboard needs more than simple totals.

A bank manager wants to know:

- How much money is flowing into the platform
- How much money is flowing out
- Whether transaction volume is increasing
- What the average transaction size is
- How customer activity changes over 7 days, 30 days, or year-to-date

Request Parameter

The endpoint accepts a timeframe:

```
7d    -> last 7 days
30d   -> last 30 days
YTD   -> from start of current year to today
```

The frontend sends this value when the admin changes the timeframe selector.

Response Data

The API returns:

- `dailyVolumes[]`
- `averageTicketSize`
- `totalInflow`
- `totalOutflow`

Meaning of Each Field

`dailyVolumes[]`

Daily transaction totals grouped by date. It allows the frontend to draw a line chart.

`averageTicketSize`

Average value of transactions during the selected timeframe.

Formula:

```
averageTicketSize = total transaction amount / number of transactions
```

`totalInflow`

Sum of CREDIT transactions.

totalOutflow

Sum of DEBIT transactions.

Backend Flow

```
Admin selects timeframe
  |
  v
Angular calls /api/admin/analytics/transactions?timeframe=30d
  |
  v
JwtInterceptor attaches ADMIN JWT
  |
  v
Spring Security validates token and role
  |
  v
AdminDashboardController receives request
  |
  v
AdminDashboardServiceImpl calculates date range
  |
  v
AdminDashboardRepository runs SUM / AVG / GROUP BY queries
  |
  v
Transaction analytics DTO returned
  |
  v
Angular line chart refreshes
```

Database Logic

The backend uses aggregation queries on the `transactions` table.

Conceptual SQL:

```
SELECT DATE(transaction_date) AS txnDate,
       SUM(CASE WHEN transaction_type = 'CREDIT' THEN amount ELSE 0 END) AS
inflow,
       SUM(CASE WHEN transaction_type = 'DEBIT' THEN amount ELSE 0 END) AS outflow
FROM transactions
WHERE transaction_date >= :startDate
GROUP BY DATE(transaction_date)
ORDER BY txnDate ASC;
```

This is efficient because the database performs the grouping and summation before sending data to Java.

Session 2: Backend Admin Loan Analytics

Endpoint

```
GET /api/admin/analytics/loans?timeframe={7d|30d|YTD}
```

This endpoint gives the admin loan analytics.

It is also protected with:

```
@PreAuthorize("hasRole('ADMIN')")
```

Response Data

The API returns:

- Loan distribution by status
- Loan distribution by product type
- NPA count
- NPA ratio

Loan Distribution

Loan applications are grouped into:

- PENDING
- APPROVED
- REJECTED

They are also grouped by product type such as:

- Personal Loan
- Home Loan
- Auto Loan
- Education Loan
- Business Loan

This allows the admin to see which loan products are most active.

NPA Count and Ratio

NPA means Non-Performing Asset.

In this project context, a loan can be treated as risky if repayments are overdue.

NPA count means:

```
number of loan accounts with overdue repayment behavior
```

NPA ratio means:

```
NPA ratio = NPA loan count / total active loan count * 100
```

Business Value

Loan analytics helps admins understand:

- Which loan products are popular
- How many applications are pending
- How many loans were approved or rejected
- Whether repayment risk is increasing
- Whether overdue loans are becoming a problem

Session 3: Angular Admin Analytics Charts

Transaction Line Chart

The admin dashboard displays a line chart:

```
X-axis -> date  
Y-axis -> amount
```

Datasets:

- Inflow
- Outflow

The chart helps the admin compare incoming and outgoing platform money movement.

Timeframe Selector

The admin can select:

- 7D
- 30D
- YTD

When the selection changes:

```
timeframe value changes  
|
```

```
      v
component calls AdminDashboardService again
      |
      v
backend returns new analytics
      |
      v
chart data is rebuilt
      |
      v
Chart.js redraws the graph
```

Loan Bar Chart

The loan analytics chart displays loan distribution by:

- Status
- Product type

Example:

```
Personal Loan: Pending 4, Approved 10, Rejected 2
Home Loan: Pending 1, Approved 5, Rejected 1
Auto Loan: Pending 3, Approved 8, Rejected 0
```

Why Charts Were Used

Tables are useful for exact data, but charts are better for pattern recognition.

The admin can quickly see:

- Inflow and outflow trends
- Loan approval pressure
- Product-wise loan demand
- Risk growth through NPA metrics

Day 43: System Audit Log

Session 1: Backend Audit Logging

What Was Implemented

Sprint 5 added audit logging so that important API activity can be tracked.

The project uses an AOP-based audit approach.

Main component:

AuditLogAspect

The aspect intercepts controller method executions and records details about API requests.

Why AOP Was Used

Audit logging is a cross-cutting concern.

That means it applies across many controllers and methods.

Without AOP, every controller method would need repeated logging code like:

```
log endpoint
log user
log status
log execution time
log error
```

Using an aspect keeps controllers clean and centralizes audit behavior in one place.

What AuditLogAspect Records

For each controller request, it records:

- Endpoint
- HTTP method
- Response status
- Execution time in milliseconds
- Acting user ID
- Timestamp
- Error message if request fails

Execution Flow

```
HTTP request enters controller
|
v
AuditLogAspect starts timer
|
v
Controller method executes
|
+-- success -> capture status and execution time
|
+-- exception -> capture sanitized error message
|
v
```

```
SystemAuditLogService saves audit record
      |
      v
Response returned to client
```

Error Capture Rule

If a controller throws an exception or returns an error response, the audit log stores a sanitized error message.

It should not store raw stack traces.

Why:

- Stack traces may reveal internal package names
- Stack traces may expose sensitive implementation details
- Admins need the error reason, not the full Java trace

Audit Log Persistence

Audit log data is persisted through:

- `SystemAuditLog` entity
- `AuditLogRepository`
- `SystemAuditLogService`

Audit writes should be isolated from the main business operation so audit tracking does not pollute business logic.

Important Technical Decision

Audit logging is separated from normal service logic.

This improves:

- Maintainability
- Consistency
- Compliance readiness
- Debugging

Session 2: Backend Audit Log Fetch API

Endpoint

```
GET /api/admin/system-logs?from=&to=&status=
```

This endpoint returns paginated and filterable system logs.

It is protected with:

```
@PreAuthorize("hasRole('ADMIN')")
```

Supported Filters

The admin can filter logs by:

- From date
- To date
- HTTP status
- Endpoint
- Method
- User

Why Pagination Is Important

Audit logs can grow very quickly.

Without pagination:

- API response becomes slow
- Browser may freeze
- Database load increases

Pagination ensures the admin loads only a manageable number of logs at once.

Backend Flow

```
Admin opens System Logs
  |
  v
Angular calls /api/admin/system-logs
  |
  v
Admin role validated
  |
  v
Controller passes filters to service
  |
  v
Service builds search criteria
  |
  v
Repository fetches paginated logs
  |
  v
DTO page returned to Angular
```

Session 3: Angular Admin System Audit Log UI

Data Grid

The audit log screen displays a searchable grid.

Columns:

- Timestamp
- Endpoint
- Method
- Status
- Execution Time
- Error

Error Rate Mini-Chart

The error rate chart shows the percentage of failed requests over time.

Failed requests include:

- 4xx client errors
- 5xx server errors

Formula:

```
errorRate = failedRequests / totalRequests * 100
```

Response Time Trend Chart

The response time chart shows how API performance changes over time.

This helps identify slow endpoints.

Business Value

Audit logs help the admin answer:

- Who performed an action?
- Which API failed?
- When did the issue happen?
- Which endpoints are slow?
- Are errors increasing?

This is important for production readiness and banking compliance.

Day 44: Advanced User Analytics - Spending and

Budgets

Session 1: Backend Spending Analytics

Endpoint

```
GET /api/analytics/spending/{userId}?months=6
```

This endpoint returns category-wise spending analytics for the authenticated user.

Security rule:

```
JWT userId must match path userId
```

If the logged-in user tries to access another user's analytics, the backend returns HTTP 403.

Response Data

The endpoint returns:

- Category spending
- Month-over-month vectors
- Budget comparison data

Category Spending

Transactions are grouped by category.

Examples:

- Bill Payment
- Recharge Payment
- Card Payment
- EMI Payment
- Insurance Payment
- Travel Payment
- Other

Month-over-Month Vector

A month-over-month vector means each category contains spending values across multiple months.

Example:

```
Category: Bill Payment  
Jan: 1200
```

```
Feb: 1500
Mar: 900
Apr: 1800
May: 1000
Jun: 1300
```

This allows the frontend to show category trends over time.

Backend Flow

```
Angular spending analytics page
  |
  v
GET /api/analytics/spending/{userId}?months=6
  |
  v
JWT user ownership check
  |
  v
InsightsServiceImpl / analytics service
  |
  v
Repository groups DEBIT transactions by category and month
  |
  v
Service pads missing months/categories with zero
  |
  v
DTO returned to Angular
```

Why Month Padding Is Needed

If a category has no spending in a month, the backend still returns zero.

This keeps the chart stable.

Without this, the chart would have uneven arrays and incorrect month alignment.

Session 2: Angular Spending Analytics UI

Doughnut Chart

The doughnut chart shows expense distribution for the selected month.

Example:

```
Bill Payment: 35%
Recharge Payment: 20%
```

```
EMI Payment: 25%  
Travel Payment: 10%  
Other: 10%
```

This helps users quickly understand where most of their money is going.

Budget vs Actual Stacked Bar Chart

The Budget vs Actual chart compares:

- Budget limit set by the user
- Actual spending from transactions

Example:

```
Food Budget: 5000, Actual: 4200  
EMI Budget: 10000, Actual: 10000  
Travel Budget: 3000, Actual: 3800
```

Month Selector

The user can change the selected month.

Flow:

```
User selects month  
  |  
  v  
Component updates selectedMonth  
  |  
  v  
Chart data is recalculated from loaded analytics  
  |  
  v  
Chart.js re-renders
```

Business Value

Spending analytics helps users:

- Identify overspending categories
- Compare actual spending against budgets
- Improve financial planning
- Understand spending patterns across months

Day 45: Advanced User Analytics - Wealth and Liabilities

Session 1: Backend Wealth Analytics

Endpoint

```
GET /api/analytics/wealth/{userId}
```

This endpoint returns wealth and liability analytics for the authenticated user.

Security rule:

```
JWT userId must match path userId
```

Response Data

The response contains:

- `netWorthTimeline[]`
- `loanPayoffForecast[]`
- `rewardAccrualHistory[]`

Net Worth Formula

Net worth is calculated as:

```
netWorth = SUM(account balances) - SUM(outstanding loan principal)
```

Example:

```
Account balances = 150000
Outstanding loan principal = 45000

netWorth = 150000 - 45000
netWorth = 105000
```

Why Outstanding Principal Is Subtracted

A loan is a liability.

Even if the user has money in their account, they still owe the remaining loan principal.

Subtracting outstanding principal gives a more realistic financial position.

Loan Payoff Forecast

The backend calculates forecast information for each active loan account.

It includes:

- Loan account number
- EMI amount
- Remaining installments
- Projected payoff date

Conceptual calculation:

```
monthsRemaining = count of unpaid repayment records  
projectedPayoffDate = due date of final unpaid repayment
```

Reward Accrual History

Reward analytics shows how points increased over time.

This helps the user understand how bill payments and transactions are contributing to rewards.

Session 2: Angular Wealth and Liability UI

Net Worth Area Chart

The frontend displays net worth progression as an area-style chart.

It compares:

- Total account balance
- Outstanding loan principal
- Net worth

This gives the user a clear picture of financial growth.

Reward Growth Line Chart

The reward chart displays reward points over time.

```
X-axis -> month  
Y-axis -> reward points
```

This shows whether rewards are increasing consistently.

Loan Payoff Forecast Chart

The loan payoff forecast shows active loan repayment progress.

It helps the user answer:

- How many months are left?
- When will this loan be fully paid?
- Which loan has the highest remaining burden?

Business Value

Wealth analytics helps customers understand their real financial position.

It combines:

- Assets
- Liabilities
- Rewards
- Loan repayment future

This makes NeoBank360 feel like a modern fintech dashboard, not only a basic banking application.

Complete Sprint 5 Backend Flow

Admin Transaction Analytics Flow

```
Admin Dashboard
  |
  v
AdminDashboardService.getTransactionAnalytics(timeframe)
  |
  v
GET /api/admin/analytics/transactions
  |
  v
Admin role check
  |
  v
Controller
  |
  v
Service calculates date range
  |
  v
Repository aggregates transactions
  |
  v
DTO returned
  |
```

v
Line chart updates

Admin Loan Analytics Flow

```
graph TD
    A[Admin Dashboard] --> B[GET /api/admin/analytics/loans]
    B --> C[Admin authorization]
    C --> D[Loan status and product aggregation]
    D --> E[NPA calculation]
    E --> F[Loan analytics DTO]
    F --> G[Bar chart updates]
```

System Audit Log Flow

```
graph TD
    A[Any API request] --> B[AuditLogAspect intercepts controller method]
    B --> C[Execution timer starts]
    C --> D[Controller/service/repository logic runs]
    D --> E[Response or exception captured]
    E --> F[Audit log saved]
    F --> G[Admin views logs in System Logs screen]
```

User Spending Analytics Flow

```
graph TD; A[User opens Insights] --> B[GET /api/analytics/spending/{userId}]; B --> C[Ownership validation]; C --> D[Transaction grouping by category and month]; D --> E[Budget vs actual calculation]; E --> F[Doughnut and bar charts render];
```

User Wealth Analytics Flow

```
graph TD; A[User opens Wealth Analytics] --> B[GET /api/analytics/wealth/{userId}]; B --> C[Ownership validation]; C --> D[Account balances loaded]; D --> E[Outstanding loans calculated]; E --> F[Net worth timeline generated]; F --> G[Reward and payoff charts render];
```

Important Technical Topics Used in Sprint 5

Backend

- Spring Boot REST APIs

- Spring Security role checks
- JWT authenticated user validation
- `@PreAuthorize`
- Spring Data JPA aggregation queries
- Native SQL grouping by date/month
- DTO projections
- BigDecimal financial calculations
- Audit logging using AOP
- Paginated audit log retrieval
- Timeframe-based analytics
- NPA risk calculation
- Read-only analytics services

Frontend

- Angular standalone components
- Angular services using HttpClient
- JWT interceptor
- Admin route guard
- Chart.js visualizations
- Theme-aware charts
- Timeframe selector
- Month selector
- Responsive dashboard cards
- Searchable admin tables
- Loading and error states

Why These Technical Decisions Matter

Why Aggregation Queries Were Used

Analytics can involve many transactions.

Loading all transactions into Angular or Java would be slow.

Using database aggregation means MySQL calculates:

- SUM
- COUNT
- AVG
- GROUP BY date
- GROUP BY category

Then Spring Boot returns only the final analytics DTO.

This improves performance and keeps APIs lightweight.

Why Audit Logging Uses AOP

Audit logging applies to many endpoints.

Instead of repeating logging code everywhere, AOP intercepts controller execution automatically.

This gives:

- Cleaner controllers
- Consistent audit format
- Easier maintenance
- Better compliance readiness

Why DTOs Were Used

DTOs prevent exposing internal entity fields.

For example:

- Password hashes are never returned
- Only required analytics fields are sent
- Frontend receives clean response structures

Why JWT Ownership Checks Were Reused

Analytics data is sensitive.

Even if a user changes the URL, the backend compares the requested user ID with the authenticated JWT user ID.

This protects financial privacy.

Business Impact

Admin Impact

Sprint 5 helps admins make operational decisions.

They can see:

- Transaction velocity
- Inflow and outflow movement
- Loan approval distribution
- NPA risk
- API failures
- Slow endpoints
- Audit history

This makes NeoBank360 closer to a real banking operations dashboard.

User Impact

Sprint 5 helps customers understand their financial health.

They can see:

- Spending by category
- Budget vs actual usage
- Net worth progression
- Loan payoff timeline
- Reward growth

This helps users make better money decisions.

Interview Explanation

If asked to explain Sprint 5, answer like this:

Sprint 5 added advanced analytics and monitoring to NeoBank360. On the admin side, I implemented transaction analytics, loan analytics, NPA tracking, and audit log monitoring. These APIs are admin-only and use optimized aggregation queries to calculate inflow, outflow, average ticket size, loan distribution, and error trends.

On the user side, I implemented advanced spending and wealth analytics. The backend groups transactions by category and month, compares actual spending with budgets, calculates net worth by subtracting outstanding loan principal from account balances, and forecasts loan payoff dates using repayment schedules.

The frontend uses Angular standalone components and Chart.js to show line charts, doughnut charts, bar charts, and wealth progression graphs. JWT interceptors attach authentication automatically, and backend ownership checks prevent users from accessing other users' analytics.

The most important engineering decision was to calculate analytics in the database using SUM, COUNT, AVG, and GROUP BY instead of loading raw records into the application. This makes the dashboards faster and more production-ready.

Interview Questions to Practice

Admin Analytics

1. How does the transaction analytics endpoint calculate total inflow and outflow?
2. Why is timeframe filtering useful in admin analytics?
3. How is average ticket size calculated?
4. How do you group transactions by date in SQL?
5. Why are admin analytics endpoints protected with `@PreAuthorize("hasRole('ADMIN')")`?
6. What is loan distribution by status?
7. What is NPA count?

8. How would you calculate NPA ratio?

Audit Logging

1. Why did you use AOP for audit logging?
2. What information is stored in the audit log?
3. Why should raw stack traces not be stored in audit logs?
4. Why is pagination important for audit logs?
5. How can audit logs help in production debugging?
6. What is a cross-cutting concern?

User Analytics

1. How is spending grouped by category?
2. Why do you return month-over-month vectors?
3. Why should missing months be returned with zero values?
4. How does Budget vs Actual analytics work?
5. How is net worth calculated?
6. Why do we subtract outstanding loan principal from account balance?
7. How is loan payoff forecast calculated?
8. How is reward growth shown to the user?

Frontend

1. How does the timeframe selector refresh admin charts?
2. How does the month selector refresh user spending charts?
3. Why are charts better than tables for analytics?
4. How do Angular services connect components to backend APIs?
5. How does the JWT interceptor support these analytics APIs?
6. How do route guards protect admin analytics pages?

Best Short Presentation Answer

Sprint 5 made NeoBank360 production-ready from an analytics and monitoring perspective. I added advanced admin analytics for transaction flow, loan distribution, and NPA risk. I also added audit logging so admin users can monitor API activity, errors, and response times. For customers, I added spending analytics, budget comparison, net worth tracking, reward growth, and loan payoff forecasting. The backend uses secured Spring Boot APIs, JWT ownership validation, admin-only role checks, and optimized database aggregation queries. The frontend uses Angular standalone components and Chart.js to present the data through modern fintech dashboards.